

PEXIP PULSE SDK · DEMONSTRATION

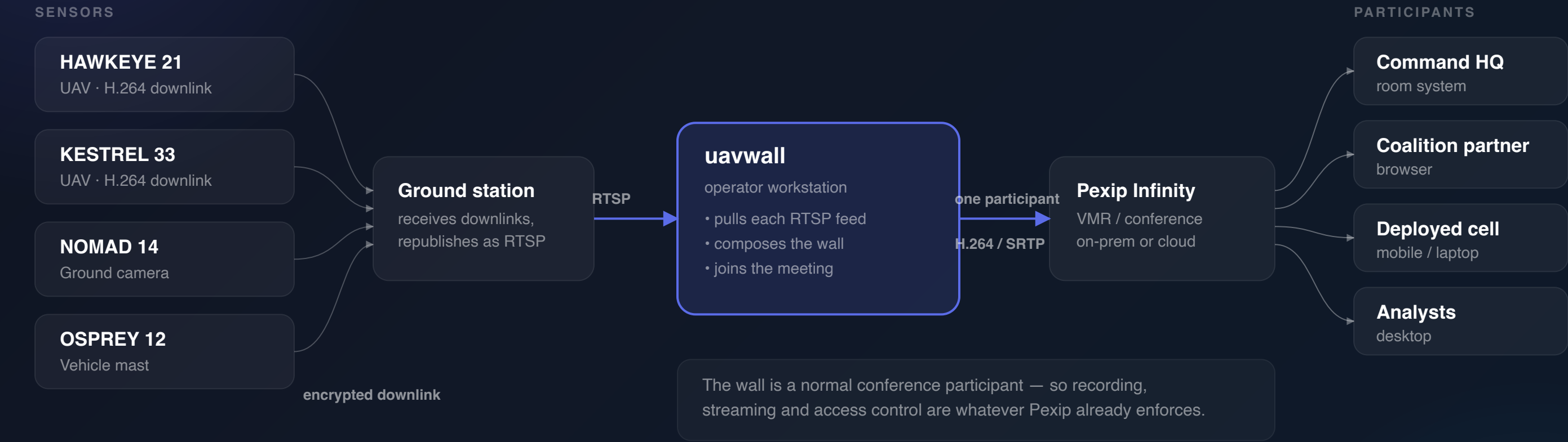
Live UAV feeds, straight into a Pexip meeting

An operator selects from live RTSP downlinks, arranges them on a wall, and that wall is pushed into a Pexip VMR — so every participant, anywhere, sees the same picture in an ordinary video call.

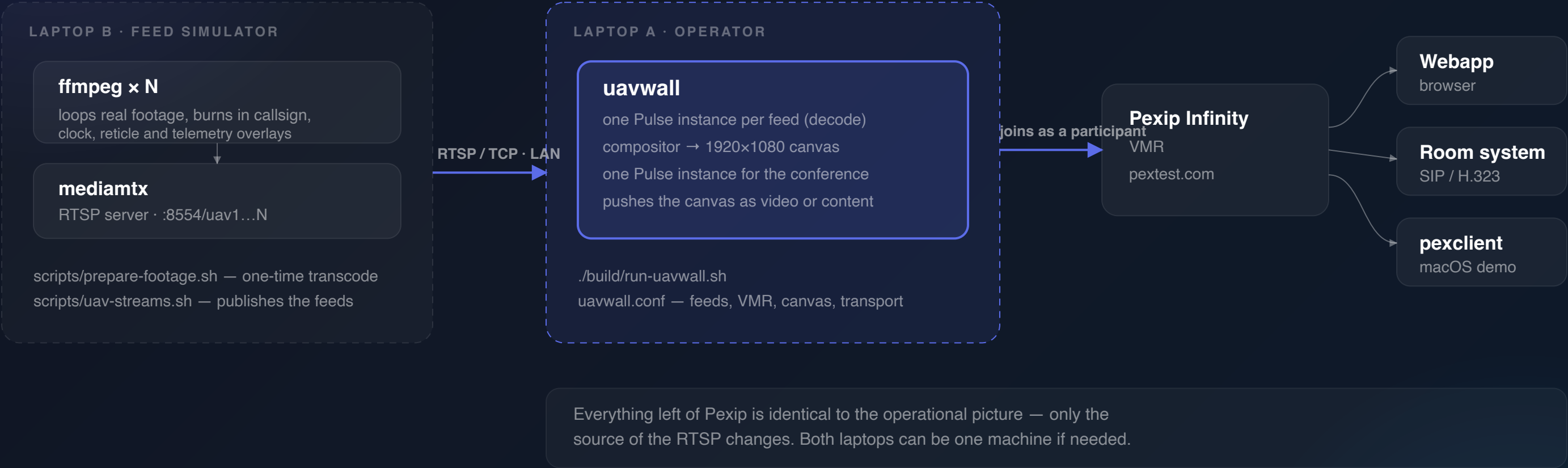
ISR video where the decisions are made

- **Any RTSP source** — UAV downlinks, ground cameras, vehicle feeds. The same protocol every IP camera already speaks.
- **Composed by an operator**, not fixed by configuration — one feed full screen, or nine at a glance, changed live.
- **Delivered as a normal Pexip participant** — no special client, no plugin. Anyone already able to join the meeting can see it.
- **Main video or content** — replace the camera, or send it on the presentation stream alongside the participants.
- **Runs on a small PC** — measured, not estimated; see the sizing slide.
- **Built on the Pulse SDK** — roughly 1,400 lines, one file.

How it fits a real deployment



How the demo is wired



Four actions, no training

1 · Connect

Feeds are listed in the rail with a live thumbnail and a status dot.
Connect all, or start the app with feeds already live.

2 · Compose

Click a feed to place it. Presets arrange the wall — 1-up, 2×2, 3×3, PiP — then drag or resize any tile.

3 · Focus

Double-click any feed to punch it full screen; double-click again to return to the previous layout. Nothing is lost.

4 · Send

Enter the VMR and send. The canvas becomes the participant's video — or its content stream — and updates live as the wall is rearranged.

Everything is live: rearranging the wall changes what the meeting sees immediately, with no renegotiation and no interruption.

What is in the demonstration

- **Any number of RTSP feeds**, named with callsigns, TCP or UDP, configurable jitter buffer.
- **Live status per feed** — receiving, stalled, failed — with the underlying error surfaced, not swallowed.
- **Layout presets and free positioning**, with full-screen punch and restore.
- **Send as main video or as content**, taking the conference floor automatically for content.
- **Send resolution and frame rate** selectable — the two levers that decide the hardware you need.
- **Live instrumentation** — per-feed resolution and frame rate, process CPU and memory, and the real transmit bitrate, packet loss and round-trip time from Pulse.
- **Reproducible benchmark** built in, so capacity can be measured on any candidate machine.
- **Self-contained demo rig** — synthetic or real footage, no aircraft and no internet required.

What it costs to run

FEEDS	CPU	MEMORY	COMPOSITE
1	45%	174 MB	13.8 ms
4	78%	279 MB	16.6 ms
9	114%	470 MB	16.9 ms

720p feeds, 1080p30 output, Apple M4 Pro.
CPU is a percentage of **one** core — 114% is 8% of that machine.

- **A fixed floor of ~45%** — compositing the canvas, almost independent of feed count.
- **~8.6% CPU and 37 MB per extra feed** — decode.
- **720p at 15fps costs 48%** where 1080p at 30fps costs 78% — indistinguishable in a 2×2 grid.

Recommended demo machine

Intel Core Ultra 5/7 or 12th-gen-plus i5/i7 mini PC, 16 GB. Single-core speed is the constraint; Quick Sync offloads decode and encode. Avoid N100-class chips.

Three commands

```
# once – normalise footage to the streaming size
./scripts/prepare-footage.sh UAV_footage

# laptop B – publish the feeds (-a prints a ready-to-paste list)
./scripts/uav-streams.sh -n 4 -d UAV_footage/prepared -a

# laptop A – the operator
./build/run-uavwall.sh
```

No aircraft, no cameras, no internet. Both roles can share one machine; running them apart keeps the measurements honest and puts the feeds on the network, as they would be in service.

Built on the Pexip Pulse SDK

One Pulse instance per feed pulls and decodes RTSP; a further instance joins the conference and accepts the composed frames. The same SDK underlying Pexip's own clients — approximately 1,400 lines of application code.